

Spring Boot Interceptors

Spring MVC interceptors allow us to execute common logic around controller execution without placing the same code inside every controller method.

They are useful when the logic depends on the **Spring MVC handler** that has been selected, such as the controller class, controller method, or annotations present on that method.

1. The Problem Interceptors Solve

Consider a controller with multiple endpoints:

```
@RestController
@RequestMapping("/api/students")
public class StudentController {

    @GetMapping
    public String getAllStudents() {
        return "All students";
    }

    @PostMapping
    public String createStudent() {
        return "Student created";
    }

    @DeleteMapping("/{id}")
    public String deleteStudent(@PathVariable Long id) {
        return "Student deleted";
    }
}
```

Suppose every request needs some common operations:

- Log the requested URL

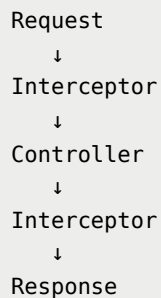
- Record execution time
- Check authentication or authorization
- Read common request information
- Identify the controller method being executed
- Add request attributes

Adding this logic directly to every controller method creates several problems:

1. Controller methods become cluttered.
2. The same code is duplicated across endpoints.
3. A small change must be repeated in several places.
4. Business logic becomes mixed with infrastructure logic.

An interceptor provides one central place for such cross-cutting MVC logic.

Definition: A Spring MVC interceptor is a component that executes before a handler, after a handler, and after the complete MVC request-processing cycle.



An interceptor does not normally replace the controller. It surrounds its execution.

2. Where an Interceptor Fits in Spring MVC

The `DispatcherServlet` is the central servlet of Spring MVC.

When a request reaches a Spring MVC application:

1. The `DispatcherServlet` receives the request.
2. A `HandlerMapping` identifies the matching handler.

3. Spring creates a handler execution chain.
4. Registered interceptors execute around the handler.
5. A `HandlerAdapter` invokes the controller method.
6. Spring processes the controller's return value.
7. The response is sent to the client.

```

Client
  ↓
Servlet Container
  ↓
DispatcherServlet
  ↓
HandlerMapping finds the handler
  ↓
Interceptor
  ↓
HandlerAdapter invokes the controller
  ↓
Response processing
  ↓
Client
  
```

Why is it called `HandlerInterceptor` ?

Spring uses the general term **handler** because the selected handler is not always a controller method. It may be:

- A controller method
- A static-resource handler
- Another handler type supported by Spring MVC

For normal controller requests, the handler is commonly represented by a `HandlerMethod`.

`HandlerExecutionChain`

When Spring finds a handler, it conceptually creates a chain containing the handler and its interceptors:

```

HandlerExecutionChain
├─ Interceptor 1
├─ Interceptor 2
├─ Interceptor 3
└─ Controller handler
  
```

The `DispatcherServlet` executes this chain. An interceptor can therefore allow or stop the request before the controller runs.

3. The Three Interceptor Lifecycle Methods

A `HandlerInterceptor` provides three main callback methods:

```

preHandle()
postHandle()
afterCompletion()
  
```

Their overall flow is:

```

Request
  ↓
preHandle()
  ↓
Controller method
  ↓
postHandle()
  ↓
Response processing or view rendering
  ↓
afterCompletion()
  ↓
Response
  
```

Method	Execution point	Main purpose
<code>preHandle()</code>	Before the selected handler executes	Validation, logging, authentication checks, starting timers
<code>postHandle()</code>	After successful handler execution, before traditional view rendering	Model or view-related post-processing

Method	Execution point	Main purpose
<code>afterCompletion()</code>	After MVC request processing finishes	Cleanup, final logging, timing, status inspection

4. Creating a Custom Interceptor

Create a Spring component that implements `HandlerInterceptor`:

```
package in.strikes.crudDemo.interceptor;

import org.springframework.stereotype.Component;
import org.springframework.web.servlet.HandlerInterceptor;

@Component
public class LoggingInterceptor implements HandlerInterceptor
{
}
```

Modern versions of `HandlerInterceptor` provide default method implementations. Therefore, an interceptor only needs to override the callbacks it actually uses.

A complete basic interceptor looks like this:

```
package in.strikes.crudDemo.interceptor;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.lang.Nullable;
import org.springframework.stereotype.Component;
import org.springframework.web.servlet.HandlerInterceptor;
import org.springframework.web.servlet.ModelAndView;

@Component
public class LoggingInterceptor implements HandlerInterceptor
{
}
```

```
@Override
public boolean preHandle(
    HttpServletRequest request,
    HttpServletResponse response,
    Object handler
) throws Exception {

    System.out.println("preHandle executed");
    return true;
}

@Override
public void postHandle(
    HttpServletRequest request,
    HttpServletResponse response,
    Object handler,
    @Nullable ModelAndView modelAndView
) throws Exception {

    System.out.println("postHandle executed");
}

@Override
public void afterCompletion(
    HttpServletRequest request,
    HttpServletResponse response,
    Object handler,
    @Nullable Exception ex
) throws Exception {

    System.out.println("afterCompletion executed");
}
}
```

For an interceptor that only checks something before the controller, overriding only `preHandle()` is sufficient:

```
@Component
public class AuthenticationInterceptor
    implements HandlerInterceptor {

    @Override
    public boolean preHandle(
        HttpServletRequest request,
        HttpServletResponse response,
        Object handler
    ) throws Exception {

        return true;
    }
}
```

5. Understanding the Method Parameters

HttpServletRequest request

Represents the incoming HTTP request.

Commonly used methods include:

```
request.getMethod();
request.getRequestURI();
request.getQueryString();
request.getHeader("Authorization");
request.getRemoteAddr();
request.getParameter("name");
request.getSession();
```

Request-specific values can also be stored as attributes:

```
request.setAttribute("requestStartTime", System.nanoTime());
```

They can later be retrieved using:

```
Long startTime =  
    (Long) request.getAttribute("requestStartTime");
```

HttpServletResponse response

Represents the outgoing HTTP response.

It can be used to set the status, headers, content type, or response content:

```
response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);  
response.setHeader("X-App-Name", "Student Management System");  
response.setContentType("application/json");  
response.getWriter().write("{  
    {  
        \"message\": \"Unauthorized\"  
    }  
}");
```

Once a response is committed, some changes may be too late. This is especially important when working with REST response bodies.

Object handler

Represents the handler selected by Spring.

For a normal controller request, it is commonly a `HandlerMethod`:

```
if (handler instanceof HandlerMethod handlerMethod) {  
  
    String controllerName =  
        handlerMethod.getBeanType().getSimpleName();  
  
    String methodName =  
        handlerMethod.getMethod().getName();  
}
```



```
System.out.println(controllerName);
System.out.println(methodName);
}
```

Possible output:

```
StudentController
getAllStudents
```

Do not cast the handler blindly:

```
HandlerMethod handlerMethod = (HandlerMethod) handler;
```

Some requests may be handled by something other than a controller method, such as a static-resource handler. Use pattern matching with `instanceof` instead:

```
if (handler instanceof HandlerMethod handlerMethod) {
    // Safe to use handlerMethod
}
```

This ability to inspect the selected controller and method is one of the major differences between an interceptor and a servlet filter.

6. Understanding `preHandle()`

Signature:

```
boolean preHandle(
    HttpServletRequest request,
    HttpServletResponse response,
    Object handler
) throws Exception
```

It executes after Spring has selected the handler but before the handler is invoked.

Why does it return `boolean` ?

The return value tells Spring whether request processing should continue.

Returning `true`

```
return true;
```

Spring continues to the next interceptor or the controller.

Returning `false`

```
return false;
```

Spring stops the execution chain, and the controller is not called.

When returning `false`, the interceptor should usually prepare an appropriate response. Otherwise, the client may receive an empty or confusing response.

```
response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
response.setContentType("application/json");
response.getWriter().write("""
    {
        "message": "Authentication required"
    }
    """);

return false;
```

Common uses of `preHandle()`

- Logging request arrival
- Starting an execution timer
- Reading a tenant or organization ID
- Creating a correlation ID
- Adding request attributes

- Validating custom headers
- Checking custom annotations
- Rejecting a request before the controller

For production-grade authentication and authorization, prefer **Spring Security** instead of building a complete security system through a custom interceptor.

7. Understanding `postHandle()`

Signature:

```
void postHandle(  
    HttpServletRequest request,  
    HttpServletResponse response,  
    Object handler,  
    ModelAndView modelAndView  
) throws Exception
```

It executes after the handler completes successfully but before traditional MVC view rendering.

```
preHandle()  
  ↓  
Controller  
  ↓  
postHandle()  
  ↓  
View rendering
```

Why does it receive `ModelAndView` ?

Traditional Spring MVC controllers may return a view. Spring stores the selected view and model data inside a `ModelAndView`.

An interceptor can add common model information:

```
@Override
public void postHandle(
    HttpServletRequest request,
    HttpServletResponse response,
    Object handler,
    ModelAndView modelAndView
) {

    if (modelAndView != null) {
        modelAndView.addObject(
            "applicationName",
            "Student Management System"
        );
    }
}
```

What happens with REST controllers?

A `@RestController` normally returns data that Spring serializes directly into the response body:

```
@RestController
public class StudentController {

    @GetMapping("/students")
    public List<Student> getStudents() {
        return studentService.getAllStudents();
    }
}
```

In such cases:

```
modelAndView == null
```

For methods using `@ResponseBody` or `ResponseEntity`, Spring may already have written the response before `postHandle()` executes. Therefore, `postHandle()` is not a reliable place to redesign or wrap a REST response body.

Use `ResponseBodyAdvice` when a common REST response transformation is required:

```
@ControllerAdvice
public class GlobalResponseAdvice
    implements ResponseBodyAdvice<Object> {
}
```

Good uses of `postHandle()`

- Updating a traditional MVC model
- Adding common view attributes
- Logging successful controller completion
- Inspecting the selected handler
- Running logic before view rendering

For REST APIs, `afterCompletion()` is often more useful for final logging and timing.

8. Understanding `afterCompletion()`

Signature:

```
void afterCompletion(
    HttpServletRequest request,
    HttpServletResponse response,
    Object handler,
    Exception ex
) throws Exception
```

It executes after the complete Spring MVC request-processing cycle finishes.

```
preHandle()
```

↓

```

Controller
  ↓
postHandle()
  ↓
Response processing or view rendering
  ↓
afterCompletion()

```

It is similar to a `finally` block and is commonly used for:

- Cleanup
- Final request logging
- Execution-time calculation
- Releasing request-related resources
- Recording the final response status
- Inspecting an unhandled exception

Example:

```

@Override
public void afterCompletion(
    HttpServletRequest request,
    HttpServletResponse response,
    Object handler,
    Exception ex
) {

    System.out.println(
        "Request completed with status: "
        + response.getStatus()
    );
}

```

Inspecting exceptions

```
@Override
public void afterCompletion(
    HttpServletRequest request,
    HttpServletResponse response,
    Object handler,
    Exception ex
) {

    if (ex != null) {
        System.out.println(
            "Request failed: " + ex.getMessage()
        );
    } else {
        System.out.println("Request completed");
    }
}
```

An important limitation is that `ex == null` does not always prove that no exception occurred.

An exception may have been handled by Spring through:

- `@ExceptionHandler`
- `@ControllerAdvice`
- `@RestControllerAdvice`
- Another exception resolver

The exception may already have been converted into a valid HTTP response. Therefore, final monitoring should also inspect:

```
response.getStatus();
```

9. Registering an Interceptor with `WebMvcConfigurer`

Creating an interceptor bean is not enough. It must also be added to Spring MVC's interceptor registry.

```
package in.strikes.crudDemo.config;

import in.strikes.crudDemo.interceptor.LoggingInterceptor;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    private final LoggingInterceptor loggingInterceptor;

    public WebConfig(LoggingInterceptor loggingInterceptor) {
        this.loggingInterceptor = loggingInterceptor;
    }

    @Override
    public void addInterceptors(
        InterceptorRegistry registry
    ) {
        registry.addInterceptor(loggingInterceptor);
    }
}
```

Role of each part

- `@Configuration` marks the class as an application configuration class.
- `WebMvcConfigurer` provides callback methods for customizing Spring MVC.
- `addInterceptors()` is used to register MVC interceptors.

- `registry.addInterceptor()` adds an interceptor to the MVC handler chain.
- Constructor injection provides the interceptor bean to the configuration class.

The `addInterceptor()` method returns an `InterceptorRegistration`, which can be used to configure:

- Included URL patterns
- Excluded URL patterns
- Execution order

10. Logging Request Details

A practical logging interceptor can access both HTTP information and the selected controller method.

```
package in.strikes.crudDemo.interceptor;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Component;
import org.springframework.web.method.HandlerMethod;
import org.springframework.web.servlet.HandlerInterceptor;

@Component
public class RequestLoggingInterceptor
    implements HandlerInterceptor {

    @Override
    public boolean preHandle(
        HttpServletRequest request,
        HttpServletResponse response,
        Object handler
    ) {

        System.out.println("----- Incoming Request -----");
        System.out.println("HTTP Method: " + request.getMetho
```

```

d());
    System.out.println("Request URI: " + request.getRequestURI());
    System.out.println("Query String: " + request.getQueryString());
    System.out.println("Client IP: " + request.getRemoteAddr());

    if (handler instanceof HandlerMethod handlerMethod) {

        String controllerName =
            handlerMethod
                .getBeanType()
                .getSimpleName();

        String methodName =
            handlerMethod
                .getMethod()
                .getName();

        System.out.println("Controller: " + controllerName);
        System.out.println("Controller Method: " + methodName);
    }

    return true;
}
}

```

`System.out.println()` is acceptable for learning examples. In a real application, use a logging framework such as SLF4J:

```

private static final Logger log =
    LoggerFactory.getLogger(RequestLoggingInterceptor.class);

```

```
ss);
```

11. Authentication Demonstration with an API Key

The following example demonstrates how an interceptor can stop a request. It is useful for understanding the flow but should not replace Spring Security in a production application.

Requirement

The client sends:

```
X-API-KEY: student-secret-key
```

- Correct key: continue to the controller
- Missing or incorrect key: return `401 Unauthorized`
- Controller must not execute

Interceptor

```
package in.strikes.crudDemo.interceptor;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.MediaType;
import org.springframework.stereotype.Component;
import org.springframework.web.servlet.HandlerInterceptor;

@Component
public class ApiKeyAuthenticationInterceptor
    implements HandlerInterceptor {

    private static final String VALID_API_KEY =
        "student-secret-key";
```

```
@Override
public boolean preHandle(
    HttpServletRequest request,
    HttpServletResponse response,
    Object handler
) throws Exception {

    String apiKey =
        request.getHeader("X-API-KEY");

    if (apiKey == null) {
        writeUnauthorizedResponse(
            response,
            "API key is missing"
        );
        return false;
    }

    if (!VALID_API_KEY.equals(apiKey)) {
        writeUnauthorizedResponse(
            response,
            "Invalid API key"
        );
        return false;
    }

    return true;
}

private void writeUnauthorizedResponse(
    HttpServletResponse response,
    String message
) throws Exception {

    response.setStatus(
        HttpServletResponse.SC_UNAUTHORIZED
    );
}
```

```

    );

    response.setContentType(
        MediaType.APPLICATION_JSON_VALUE
    );

    response.getWriter().write(
        """
        {
            "message": "%s"
        }
        """.formatted(message)
    );
}
}

```

The response is written before returning `false` because the controller will not execute and will not create a response.

Registering it for selected paths

```

@Configuration
public class WebConfig implements WebMvcConfigurer {

    private final ApiKeyAuthenticationInterceptor
        authenticationInterceptor;

    public WebConfig(
        ApiKeyAuthenticationInterceptor authenticationInt
        erceptor
    ) {
        this.authenticationInterceptor =
            authenticationInterceptor;
    }

    @Override

```

```

public void addInterceptors(
    InterceptorRegistry registry
) {
    registry.addInterceptor(authenticationInterceptor)
        .addPathPatterns("/api/**")
        .excludePathPatterns(
            "/api/auth/login",
            "/api/public/**"
        );
}
}

```

12. Authorization Demonstration

Suppose only users with the `ADMIN` role can access a delete operation.

The client sends:

```
X-USER-ROLE: ADMIN
```

A simplified interceptor check could be:

```

@Override
public boolean preHandle(
    HttpServletRequest request,
    HttpServletResponse response,
    Object handler
) throws Exception {

    if ("DELETE".equalsIgnoreCase(request.getMethod())) {

        String role =
            request.getHeader("X-USER-ROLE");

        if (!"ADMIN".equals(role)) {

```

```

        response.setStatus(
            HttpServletResponse.SC_FORBIDDEN
        );

        response.setContentType(
            MediaType.APPLICATION_JSON_VALUE
        );

        response.getWriter().write(
            """
            {
                "message": "Admin access required"
            }
            """
        );

        return false;
    }

    return true;
}

```

This is a learning example. Real authorization rules should normally be implemented through Spring Security.

401 VS 403

Status	Meaning	Example
401 Unauthorized	The client has not been successfully authenticated	Missing, invalid, or expired token
403 Forbidden	The client is authenticated but lacks permission	A normal user attempts an admin-only operation

```
HttpServletResponse.SC_UNAUTHORIZED // 401
HttpServletResponse.SC_FORBIDDEN   // 403
```

13. Measuring Request Execution Time

The basic requirement is:

```
Record start time
    ↓
Execute the controller
    ↓
Calculate total duration
```

Incorrect approach: using an instance field

```
@Component
public class TimingInterceptor
    implements HandlerInterceptor {

    private long startTime;

    @Override
    public boolean preHandle(
        HttpServletRequest request,
        HttpServletResponse response,
        Object handler
    ) {

        startTime = System.currentTimeMillis();
        return true;
    }
}
```

This is unsafe because Spring components are singleton-scoped by default. Multiple requests may use the same interceptor instance concurrently.

Request A starts → startTime = 1000
Request B starts → startTime = 1100
Request A ends → incorrectly uses 1100

Request-specific mutable data should not be stored in a normal interceptor instance field.

Correct approach: use request attributes

Each request has its own `HttpServletRequest` object. Store the starting time in the request:

```
request.setAttribute(  
    "requestStartTime",  
    System.nanoTime()  
);
```

Retrieve it in `afterCompletion()`:

```
Long startTime =  
    (Long) request.getAttribute("requestStartTime");
```

Complete timing interceptor

```
package in.strikes.crudDemo.interceptor;  
  
import jakarta.servlet.http.HttpServletRequest;  
import jakarta.servlet.http.HttpServletResponse;  
import org.springframework.stereotype.Component;  
import org.springframework.web.method.HandlerMethod;  
import org.springframework.web.servlet.HandlerInterceptor;  
  
import java.util.concurrent.TimeUnit;  
  
@Component
```

```

public class RequestTimingInterceptor
    implements HandlerInterceptor {

    private static final String START_TIME_ATTRIBUTE =
        "requestStartTime";

    @Override
    public boolean preHandle(
        HttpServletRequest request,
        HttpServletResponse response,
        Object handler
    ) {

        request.setAttribute(
            START_TIME_ATTRIBUTE,
            System.nanoTime()
        );

        return true;
    }

    @Override
    public void afterCompletion(
        HttpServletRequest request,
        HttpServletResponse response,
        Object handler,
        Exception ex
    ) {

        Long startTime =
            (Long) request.getAttribute(
                START_TIME_ATTRIBUTE
            );

        if (startTime == null) {
            return;
        }
    }
}

```

```

    }

    long durationNanos =
        System.nanoTime() - startTime;

    long durationMillis =
        TimeUnit.NANOSECONDS.toMillis(
            durationNanos
        );

    String handlerName = "UnknownHandler";

    if (handler instanceof HandlerMethod handlerMethod) {
        handlerName =
            handlerMethod
                .getBeanType()
                .getSimpleName()
                + "."
                + handlerMethod
                .getMethod()
                .getName();
    }

    System.out.println(
        "Request: "
            + request.getMethod()
            + " "
            + request.getRequestURI()
    );

    System.out.println(
        "Handler: " + handlerName
    );

    System.out.println(
        "Status: " + response.getStatus()
    );

```

```
);

    System.out.println(
        "Execution time: "
        + durationMillis
        + " ms"
    );
}
}
```

Why use `afterCompletion()` ?

`postHandle()` runs after successful handler execution.

`afterCompletion()` measures closer to the end of the complete MVC lifecycle, including later processing such as exception resolution and traditional view handling.

Why use `System.nanoTime()` ?

- `System.currentTimeMillis()` represents wall-clock time.
- The system clock can be adjusted.
- `System.nanoTime()` is designed for measuring elapsed time.

14. Applying an Interceptor to URL Patterns

Without path configuration:

```
registry.addInterceptor(loggingInterceptor);
```

the interceptor may apply broadly to requests handled by Spring MVC's configured handler mappings.

One URL pattern

```
@Override
public void addInterceptors(
    InterceptorRegistry registry
) {
    registry.addInterceptor(loggingInterceptor)
        .addPathPatterns("/api/students/**");
}
```

Multiple URL patterns

```
registry.addInterceptor(loggingInterceptor)
    .addPathPatterns(
        "/api/students/**",
        "/api/teachers/**",
        "/api/courses/**"
    );
```

Common patterns

Pattern	Meaning
<code>/api/students</code>	Exact path
<code>/api/students/*</code>	One path segment after <code>/students</code>
<code>/api/students/**</code>	Any nested path under <code>/students</code>
<code>/api/**</code>	All API paths
<code>/**</code>	All matching MVC paths

15. Excluding URL Patterns

Suppose authentication should apply to all API routes except login and registration:

```
@Override
public void addInterceptors(
```

```

        InterceptorRegistry registry
    ) {
        registry.addInterceptor(authenticationInterceptor)
            .addPathPatterns("/api/**")
            .excludePathPatterns(
                "/api/auth/login",
                "/api/auth/register"
            );
    }

```

Included and excluded patterns can be combined on the same interceptor registration.

16. Multiple Interceptors and Execution Order

Suppose two interceptors, **A** and **B**, are registered in that order.

Their successful execution flow is:

```

A.preHandle()
  ↓
B.preHandle()
  ↓
Controller
  ↓
B.postHandle()
  ↓
A.postHandle()
  ↓
Response processing or view rendering
  ↓
B.afterCompletion()
  ↓
A.afterCompletion()

```

The pattern is:

- `preHandle()` executes in forward order.
- `postHandle()` executes in reverse order.
- `afterCompletion()` executes in reverse order.

This resembles nested method calls:

```
A opens
  B opens
    Controller executes
  B closes
A closes
```

Configuring the order

```
@Override
public void addInterceptors(
    InterceptorRegistry registry
) {

    registry.addInterceptor(loggingInterceptor)
            .order(1);

    registry.addInterceptor(authInterceptor)
            .order(2);

    registry.addInterceptor(timingInterceptor)
            .order(3);
}
```

Lower order values have higher priority and execute earlier during `preHandle()`.

Only interceptors whose `preHandle()` successfully participated in the chain are eligible for the corresponding completion callbacks.

17. Filters and Interceptors Together

A filter surrounds servlet processing. An interceptor surrounds a Spring MVC handler.

```

HTTP Request
  ↓
Servlet Container
  ↓
Filter 1: before chain.doFilter()
  ↓
Filter 2: before chain.doFilter()
  ↓
DispatcherServlet
  ↓
HandlerMapping finds controller
  ↓
Interceptor 1: preHandle()
  ↓
Interceptor 2: preHandle()
  ↓
Controller
  ↓
Interceptor 2: postHandle()
  ↓
Interceptor 1: postHandle()
  ↓
Response processing or view rendering
  ↓
Interceptor 2: afterCompletion()
  ↓
Interceptor 1: afterCompletion()
  ↓
Filter 2: after chain.doFilter()
  ↓
Filter 1: after chain.doFilter()
  ↓
HTTP Response
  
```

Core difference

- A **filter** works at the Servlet layer.
- An **interceptor** works inside Spring MVC after a handler has been selected.

Request and response wrapping

A filter can wrap or replace the request and response objects passed further into the chain:

```
chain.doFilter(wrappedRequest, wrappedResponse);
```


This makes filters suitable for:

- Caching a request body
- Reading a request body multiple times
- Capturing a response body
- Transforming request or response data
- Compression and encoding concerns

An interceptor receives the request and response after processing has already entered Spring MVC. It cannot replace the objects used by the entire servlet chain in the same way.

18. Filter vs Interceptor

Point	Filter	Interceptor
Layer	Servlet layer	Spring MVC layer
Main target	Servlet request and response	Selected MVC handler
Knows controller method	Normally no	Yes, through <code>HandlerMethod</code>
Main before callback	Code before <code>chain.doFilter()</code>	<code>preHandle()</code>
Main after callback	Code after <code>chain.doFilter()</code>	<code>postHandle()</code> and <code>afterCompletion()</code>
Can wrap request/response	Yes	Not in the same chain-wide manner
Can block processing	Yes	Yes, by returning <code>false</code> from <code>preHandle()</code>
Suitable for body wrapping	Yes	No
Suitable for controller annotations	No	Yes
Common security mechanism	Spring Security filter chain	Not a replacement for Spring Security

19. When to Use a Filter

Use a filter when the concern applies to the complete HTTP or servlet request rather than specifically to a controller method.

Common examples:

1. Request or response body wrapping
2. Request-body caching
3. Response-body capture or transformation
4. Character encoding
5. Compression
6. CORS-related processing
7. Logging all servlet traffic
8. Security processing before Spring MVC

Spring Security uses servlet filters because authentication and authorization generally need to happen before a request reaches the controller layer.

20. When to Use an Interceptor

Use an interceptor when the requirement depends on Spring MVC or the selected handler.

Common examples:

1. Logging controller and method names
2. Reading custom annotations from controller methods
3. Starting and completing controller-related timing
4. Adding common request attributes
5. Adding common model attributes in traditional MVC
6. Collecting MVC-specific metrics
7. Applying tenant or organization context based on handler rules
8. Running logic before and after a particular group of controller endpoints

21. Common Mistakes

Mistake 1: Creating an interceptor but not registering it

`@Component` creates the bean, but `addInterceptors()` places it in the Spring MVC chain.

Mistake 2: Returning `false` without writing a response

When `preHandle()` returns `false`, the controller does not execute. Prepare an appropriate status and body before stopping the chain.

Mistake 3: Storing request data in an interceptor field

Interceptors are normally singleton beans. Store request-specific values in request attributes instead.

Mistake 4: Blindly casting `handler`

Not every handler is a `HandlerMethod`. Always check with `instanceof`.

Mistake 5: Modifying a REST body inside `postHandle()`

The response may already have been written. Use `ResponseBodyAdvice` or a filter-based wrapper, depending on the requirement.

Mistake 6: Building production security manually

A custom API-key interceptor is useful for learning request flow, but production authentication and authorization should normally use Spring Security.

Mistake 7: Assuming `ex == null` means no error occurred

A global exception handler may already have handled the exception. Also inspect the final HTTP status.

22. Final Mental Model

A servlet filter asks:

What should happen around the complete HTTP request before or after it reaches a servlet?

A Spring MVC interceptor asks:

What should happen around the selected Spring MVC handler before and after the controller executes?

Filter → surrounds DispatcherServlet and servlet processing

Interceptor → surrounds the selected Spring MVC handler

The most important lifecycle rule is:

```
preHandle()      → before the controller
postHandle()     → after successful controller execution
afterCompletion() → after MVC request processing finishes
```

And with multiple interceptors:

```
Forward order before the controller
Reverse order after the controller
```